INVENTORY MANAGEMENT SYSTEM

Advanced Java Collections Project Documentation

1️⃣ Project Overview

The Inventory Management System is a console-based Java application designed to manage product inventory efficiently using Java Collections Framework.

This project demonstrates:

- Advanced use of collections
- Data structure selection based on performance
- Comparable and Comparator interfaces
- Undo functionality using Stack
- Transaction tracking using LinkedList
- Low stock management using Queue
- Generic utility methods

2️⃣ Objectives

The main objectives of this project are:

- Store products uniquely using HashSet
- Maintain transaction history
- Implement undo functionality
- Provide multiple sorting options
- Manage low stock alerts
- Apply real-world data structure concepts

3️⃣ Technologies Used

- Java (JDK 8+)
- Java Collections Framework
- OOP Concepts
- Comparable & Comparator

- Generics

4️⃣ Project Structure

```
InventoryManagementSystem/
│
├── Main.java
├── Product.java
├── Transaction.java
├── InventoryUtils.java
├── InventoryManagementSystem.java
├── PriceComparator.java
├── NameComparator.java
└── ValueComparator.java
```

5️⃣ Class Descriptions

🔹 Product.java

Represents a product in inventory.

Attributes:

- SKU
- Name
- Category
- Price
- Quantity

Implements:

- Comparable<Product> (Natural sorting by SKU)

Key Methods:

- getValue()
- compareTo()
- equals()
- hashCode()

🔹 InventoryManagementSystem.java

Core business logic class.

Collections Used:

- HashSet<Product> → Unique product storage
- LinkedList<Transaction> → Transaction history
- Stack<Runnable> → Undo operations
- Queue<Product> → Low stock alerts

Key Features:

- Add Product
- Update Quantity
- View Sorted Products
- Undo Last Operation
- Show Transactions
- Show Low Stock Alerts
- Display Inventory Statistics

🔹 Transaction.java

Stores transaction details:

- Type (ADD / UPDATE)
- Description
- Timestamp

Used to track inventory changes.

🔹 InventoryUtils.java

Utility class with static generic methods:

- findBySku()
- calculateTotalValue()

🔹 Comparator Classes

1. PriceComparator → Sort by price (descending)

2. NameComparator → Sort alphabetically
3. ValueComparator → Sort by total inventory value

All implement:

Comparator<Product>

6️⃣ Data Structures Used & Justification

🔹 HashSet

Purpose:

- Store unique products
- Fast lookup by SKU

Time Complexity:

- Add: O(1)
- Search: O(1)

🔹 LinkedList

Purpose:

- Maintain transaction history
- Fast insertion at beginning

Time Complexity:

- Insert at head: O(1)

🔹 Stack

Purpose:

- Undo functionality
- Follows LIFO principle

Time Complexity:

- Push/Pop: O(1)

🔹 Queue

Purpose:

- Manage low stock alerts
- FIFO processing

Time Complexity:

- Offer/Poll: O(1)

7️⃣ System Workflow

Add Product

1. Product added to HashSet
2. Transaction recorded
3. Undo operation stored in Stack
4. Low stock checked

Update Quantity

1. Product found using SKU
2. Quantity updated
3. Transaction recorded
4. Undo action stored
5. Low stock evaluated

Undo Operation

1. Last action popped from Stack
2. Reverse operation executed

8️⃣ Time Complexity Summary

Operation	Complexity
Add Product	O(1)
Search Product	O(1)
Update Quantity	O(1)
View Sorted Products	O(n log n)

Operation	Complexity
Undo	O(1)
Add Transaction	O(1)

---

9 Key Concepts Demonstrated

- Comparable vs Comparator
- Generic methods
- Hash-based collections
- Linked structures
- LIFO & FIFO principles
- OOP encapsulation
- Collection sorting

---

10 Limitations

- No database persistence
- Console-based UI
- No multi-user support
- No file storage

---

11 Future Enhancements

- Add Redo functionality
- Add file storage (Save/Load)
- Convert to GUI (JavaFX/Swing)
- Add database integration
- Implement multi-threading
- Add search filters

---

12 Conclusion

This project demonstrates practical implementation of:

- Java Collections Framework
- Data structure optimization
- Object-Oriented Design
- System architecture planning